

DPO on a Colab T4

Verdict: DON'T SHIP. This is a reproducible streaming experiment, not evidence of useful Russian support behavior. Soup 0.75.2 trained Qwen2.5-0.5B-Instruct at model revision 7ae5576, training commit 55bf7f4; verification retry uses b21a349. Transformers 5.16.1, TRL 0.29.0, PEFT 0.20.0 and PyTorch 2.11.0+cu128 ran on a Tesla T4 (15,360 MiB), Python 3.12.13. One epoch uses 400 pairs, batch 1, accumulation 4, beta 0.1, LR 2e-5, seed 42, fp16 base, fp32 q/v LoRA (r=8), two pinned RAM buffers, and 512 total tokens.

The 500 pairs come from pinned d0rj/r lhf-reward-datasets-ru, a translated general preference corpus. I retained its chosen/rejected labels, stripped transcript markers, filtered unsuitable rows and deterministically split unique normalized prompts 400/50/50. The original source test is English, so it was excluded. Project prompt overlap is zero; semantic overlap and label correctness remain unproven. The eight task checks are clearly labeled synthetic functional fixtures, not support-ticket evidence.

1. Memory budget, calculated before training

With H=896, I=4,864, V=151,936 and 24 layers, one decoder layer has 14,912,384 parameters. DPO concatenates chosen/rejected into two rows. Accumulation retains gradients, not four activation graphs.

Consumer and calculation	Estimated allocation
Tied vocabulary matrix: $V \times H \times 2$ bytes; two layer buffers: $2 \times 14,912,384 \times 2$	259.66 + 56.89 MiB
540,672 LoRA parameters $\times$ 16 bytes (FP32 weights, grads, Adam m/v); frozen reference $\times$ 4	8.25 + 2.06 MiB
Checkpoint boundaries: $2 \times 512 \times H \times 25 \times 2$; within-layer MLP allowance: $2 \times 512 \times I \times 2 \times 8$	43.75 + 76 MiB
Logits: $2 \times 512 \times V$; fp16 storage / fp32 loss operations	296.75 / 593.50 MiB
Logits/loss envelope: 14 bytes per element; CUDA/allocator allowance	2.029 + 1 GiB
Total planned VRAM; CPU decoder store; full fp16 checkpoint	3.465 GiB; 682.63 MiB; 942.29 MiB

The reference shares the base and has a frozen initial LoRA; its sequential pass still allocates logits. Measured training peaks: **2.506 GiB allocated, 6.340 GiB reserved, 6,657 MiB sampled driver memory**; host RSS reached 3.25 GiB. Live allocation was 0.958 GiB below plan: actual sequences reached 419 rather than 512 tokens, and the workspace charge was conservative. Driver peak exceeded plan by 3.036 GiB, mostly allocator reservations rather than live tensors; variable batch lengths plausibly retained differently sized blocks. End-of-step live memory stayed about 0.336 GiB, arguing against a live-tensor leak. The 1 GiB allocator allowance was inadequate: budget at least 8 GiB total device memory for this replay, then remeasure changed shapes. Host RSS also includes imports, sharding, pinned copies, tokenizer/data and telemetry; the CPU store estimate is not total process RAM.

2. Did the saved model change meaningfully?

Loss went from 0.693147 to 0.683964 over 100 attempted steps: **97 updates, 3 AMP skips** (1, 2, 33). Falling loss can reflect batch difficulty, leakage, wrong labels, drifting reference or partial gradients. `verify_adapter.py` requires changed B tensors, exact fresh PEFT reload and adapter-on/off effects above repeat noise. On 50 held-out pairs, completion-only mean margin gain was **0.369583**, bootstrap 95% lower bound **0.207135** (gates ≥ 0.05 and > 0). All adapter checks passed; an unchanged checkpoint was rejected. The separate validation rows were prepared but not evaluated during training. The four-layer streamed/resident fixture matched logits, DPO loss, all 16 gradients and post-AdamW adapter states exactly; its zero-gradient control failed. These checks do not prove full 0.5B backward equivalence, correct labels, semantic novelty or support-domain quality.

3. Silent failures and the checks that cover them

Risk	What was checked	What Soup establishes
Missing, dead or wrongly saved LoRA; incorrect streamed gradients	Trainable storage, all 24 layers' successful-update gradients, changed B tensors, exact fresh reload; four-layer resident/streamed forward, loss, gradients and identical AdamW update	Setup guards meta adapters. Doctor/dry-run do not establish gradient or saved-artifact correctness. The small parity fixture does not prove full 0.5B backward equivalence.
Reference drifts or equals evolving policy	Initial reference tensors frozen; tensor drift measured after training; probe changes policy while reference matches resident base	Current TRL/Soup materializes a frozen reference LoRA. A lower loss alone cannot verify it.
Wrong template, truncation or completion mask	Conversational input, prepared IDs, nonempty/distinct completions, EOS retention, actual collator shapes; scoring uses training's keep_start prompt cap	Data doctor refuses DPO. Its chosen-only chat projection scans 200/400 rows and warns about missing generation markers; it is an SFT mask preview. Dry-run ends before real streaming preflight.
Leakage, length shortcuts or bad labels	Normalized prompt split checks and all 400 pairs linted; length bias $d=0.357$, chosen longer in 59.8%; source provenance retained	Lint reports MINOR length bias. It labels the skipped near-duplicate check OK when datasketch is absent. It cannot establish preference truth or support-domain quality.
Misinterpreted AMP overflows or mismatched ship precision	Raw scaled overflows, unscaled gradients and actual skipped updates recorded; both ship arms evaluated in fp16 with raw response journals	AMP can skip an overflow safely. Soup's live ship selects bf16 on this T4; measured fp16 scores are supplied to its offline gate instead.

4. Failures, final verdict and required changes

Attempt 01 failed on Colab's unused TorchAO 0.10 versus PEFT's ≥ 0.16 probe; uninstalling it fixed plain LoRA. Attempt 02 failed on symlinked Hub weights; a checksum-verified regular-file copy fixed setup. Attempt 03 reached step 33, then my logger rejected an infinite AMP-scaled gradient. I fixed serialization and checked unscaled gradients belonging to successful updates. Attempt 04 trained, but my verifier misread Transformers' default BatchEncoding return type. Explicit TRL-compatible return types fixed retry 05 without retraining or changing the saved adapter. All failed checks, malformed telemetry and the notebook's failed paste remain intact. Attempts 01-03 had misleading outer exit zero. Attempt 04 correctly remains incomplete; retry 05 completes the missing checks separately. `pip` check's missing-Jedi advisory for Colab IPython is preserved.

DON'T SHIP: the adapter changed and improved held-out preference margins, and training fit the T4, but measured fp16 soup ship rejected the task tie: **4/8 correct for both base and tuned**. Across 270 bundled general-suite items, arithmetic rose 35/36 to 36/36 and over-refusal 38/40 to 39/40; other suites tied, with no regression flagged at the 0.05 absolute-score tolerance. These small checks cannot establish support quality. First obtain reviewed Russian support preferences, split by ticket/template family, add held-out policy/grounding/escalation tasks and broader regressions, then retrain and demonstrate a strict task win alongside the frozen adapter and memory checks. Retain full-size backward parity as an unresolved streaming correctness concern; expand that check before relying on a larger deployment model.

What surprised me and still concerns me

I was surprised that the Russian dataset's official test split was English, and that a skipped near-duplicate check could be printed as OK. The AMP logging failure also showed how an observability bug can be mistaken for a training failure. My main concern remains the gap between learning these translated preferences and answering real support tickets correctly. Neither the small parity fixture nor the mini benchmark suites close that gap.

Evidence: `artifacts/attempt-04/` holds timestamped/raw training and ship logs, `nvidia-smi`, memory, gradients and reference checks; `attempt-05/` holds the successful saved-adapter verification and rejected control. Failures are retained alongside them. Method and pinned upstream sources: [verification contract](#), [Soup source audit](#), [data manifest](#), [DPO paper](#), [pinned Qwen config](#).